

DCE533 · COMPUTAÇÃO GRÁFICA

Pipeline Gráfico e Projeções

Visualização · Projeções · Frustum e NDC · Recorte · Viewport

Prof. Paulo Bressan

UNIFAL-MG — Bacharelado em Ciência da Computação



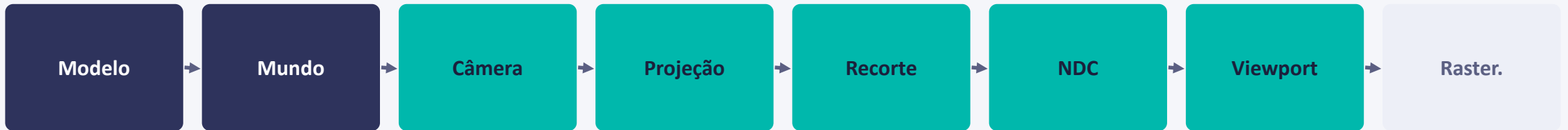
O que vamos ver hoje

1	Abertura & revisão do pipeline	10 min
2	Transformação de visualização	25 min
3	Projeções: ortográfica e perspectiva	20 min
4	Frustum e coordenadas NDC	15 min
5	Recorte (Clipping)	18 min
6	Transformação de Viewport	10 min

Convenção: vetor-linha — $P' = P \cdot M$

Os pontos são linhas; nas composições, o fator mais à esquerda é aplicado primeiro.

Onde estamos no pipeline gráfico?



azul-escuro = já visto teal = tema desta aula cinza = próxima aula

- Já sabemos posicionar e transformar objetos dentro do mundo (aula anterior)
- Hoje: levar a cena do mundo 3D até a tela 2D do usuário
- Cinco etapas novas — cada uma é uma matriz, e todas se compõem numa só

$$P_{\text{tela}} = P_{\text{mundo}} \cdot V \cdot P \cdot W$$

Os espaços de coordenadas e a nossa cena

Mundo

Todos os objetos numa cena comum

Câmera

Tudo reexpresso do ponto de vista do observador

Recorte / NDC

Cubo canônico $[-1,1]^3$, independente da resolução

Tela

Pixels reais da janela, prontos para rasterizar

A cena do exercício

P_{mundo}

(0, 2, -5) — o ponto que vamos acompanhar

eye

(4, 0, 3) — fora dos eixos coordenados

target

(0, 0, 0) — para onde a câmera olha

up

(0, 1, 0) — referência de vertical

n

1 — distância do plano near

FOV_y

90° — abertura vertical da câmera

viewport

800 × 600 pixels

Fio condutor da aula

Um único ponto atravessa todo o pipeline, calculado no quadro a cada etapa, até virar um pixel exato.

01

PARTE 1 DE 4

Transformação de Visualização

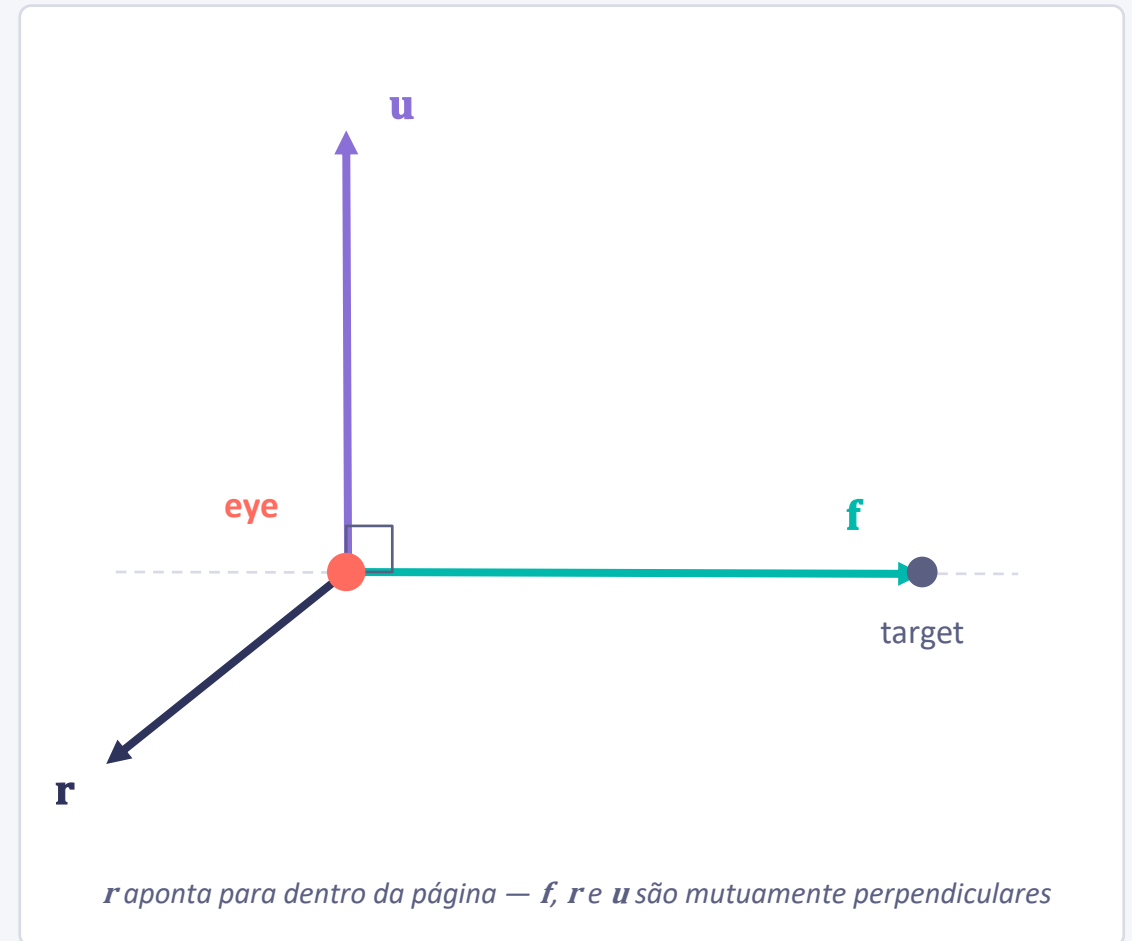
Construindo a base da câmera e reexpressando a cena a partir dela

Três parâmetros definem o olhar

- A câmera é descrita por onde está, para onde olha e o que considera vertical
- Desses três dados nasce uma base ortonormal própria da câmera

onde

eye	posição da câmera no mundo
target	ponto para onde ela aponta
up	vertical de referência do mundo
f	frente — direção de visão
r	lateral — direita da câmera
u	vertical real da câmera



Construindo a base ortonormal

Fórmulas

$$\mathbf{f} = (\text{target} - \text{eye}) / \|\text{target} - \text{eye}\|$$

$$\mathbf{r} = (\mathbf{f} \times \text{up}) / \|\mathbf{f} \times \text{up}\|$$

$$\mathbf{u} = \mathbf{r} \times \mathbf{f}$$

onde

$\times$	produto vetorial
$\ \cdot\ $	norma (comprimento) do vetor
$/ \ \cdot\ $	normalização — deixa o vetor unitário

EXERCÍCIO

$$\text{target} - \text{eye} = (-4, 0, -3) \quad \|\cdot\| = 5$$

$$\mathbf{f} = (-4, 0, -3) / 5 = (-0.8, 0, -0.6)$$

$$\mathbf{f} \times \text{up} = (0.6, 0, -0.8) \quad \|\cdot\| = 1$$

$$\mathbf{r} = (0.6, 0, -0.8)$$

$$\mathbf{u} = \mathbf{r} \times \mathbf{f} = (0, 1, 0)$$

Do mundo para o espaço da câmera

- Converter um ponto é projetá-lo sobre cada eixo da nova base
- Antes disso, subtrai-se eye — é o que leva a câmera à origem

$$\mathbf{d} = \mathbf{P}_{\text{mundo}} - \text{eye}$$

$$x_c = \mathbf{d} \cdot \mathbf{r} \quad y_c = \mathbf{d} \cdot \mathbf{u} \quad z_c = \mathbf{d} \cdot \mathbf{f}$$

onde

$\mathbf{d}$ vetor de eye até o ponto

$\cdot$ produto escalar

x_c, y_c, z_c coordenadas no espaço da câmera

EXERCÍCIO

$$\mathbf{d} = (0, 2, -5) - (4, 0, 3) = (-4, 2, -8)$$

$$\begin{aligned} x_c &= (-4)(0.6) + (2)(0) + (-8)(-0.8) \\ &= -2.4 + 6.4 = 4 \end{aligned}$$

$$\begin{aligned} y_c &= (-4)(0) + (2)(1) + (-8)(0) \\ &= 2 \end{aligned}$$

$$\begin{aligned} z_c &= (-4)(-0.8) + (2)(0) + (-8)(-0.6) \\ &= 3.2 + 4.8 = 8 \end{aligned}$$

$$\mathbf{P}_{\text{câmera}} = (4, 2, 8)$$

A matriz de visualização V

Forma simbólica (vetor-linha)

$$V = \begin{bmatrix} r_x & u_x & f_x & 0 \\ r_y & u_y & f_y & 0 \\ r_z & u_z & f_z & 0 \\ -e \cdot r & -e \cdot u & -e \cdot f & 1 \end{bmatrix}$$

onde

V	matriz de visualização 4×4
colunas	guardam r , u e f
última linha	guarda a translação ($-eye$)

EXERCÍCIO

$$V = \begin{bmatrix} 0.6 & 0 & -0.8 & 0 \\ 0 & 1 & 0 & 0 \\ -0.8 & 0 & -0.6 & 0 \\ 0 & 0 & 5 & 1 \end{bmatrix}$$

$$(0, 2, -5, 1) \cdot V = (4, 2, 8, 1)$$

mesmo resultado dos produtos escalares

02

PARTE 2 DE 4

Projeções

Como uma cena tridimensional vira uma imagem plana

Ortográfica vs. perspectiva

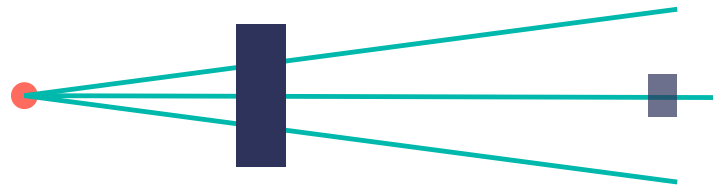
Ortográfica (paralela)

- Raios de projeção paralelos entre si
- Preserva paralelismo e medidas
- CAD, plantas técnicas, vistas ortogonais

Ortográfica

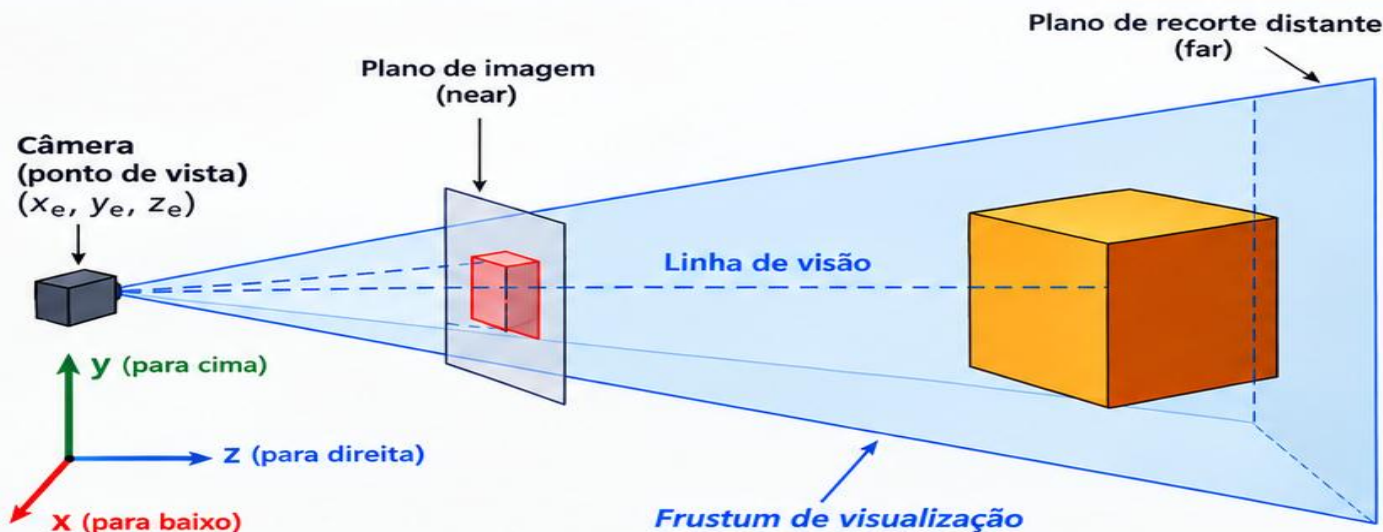


Perspectiva



Sistema de Projeção Perspectiva (Cônica)

A projeção perspectiva simula a visão humana, onde as linhas paralelas convergem em um ponto de fuga.



Parâmetros da Visualização

Ponto de vista (eye / câmera)

Posição da câmera no mundo 3D.

(x_e, y_e, z_e)

Ponto de referência (center / look-at)

Ponto para o qual a câmera está apontada.

(x_c, y_c, z_c)

Vetor up

Define a orientação vertical da câmera.

(geralmente $(0, 1, 0)$)

Campo de visão (FOV – field of view)

Ângulo de abertura da câmera (vertical ou horizontal).

(ex.: 60°)

Razão de aspecto (aspect ratio)

Relação entre a largura e a altura da janela de visualização.

(ex.: 16:9)

Plano próximo (near)

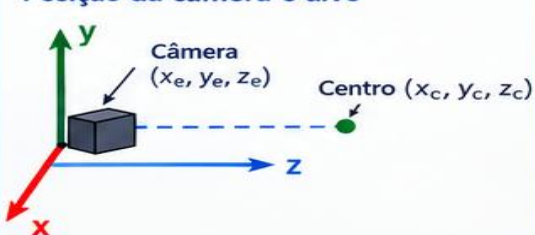
Distância do plano de recorte próximo à câmera. ($z_{near} > 0$)

Plano distante (far)

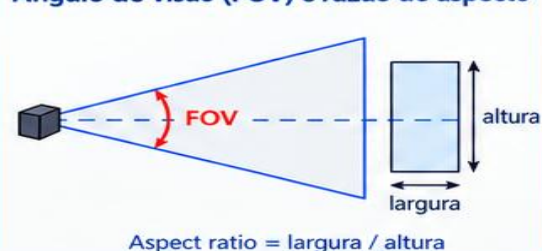
Distância do plano de recorte distante à câmera. ($z_{far} > z_{near}$)

Parâmetros da Visualização (visão geral)

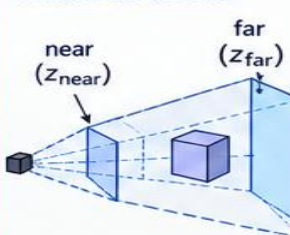
Posição da câmera e alvo



Ângulo de visão (FOV) e razão de aspecto



Planos de recorte



Parâmetros do sistema de coordenadas

Eixo y: para cima

Eixo z: para direita

Eixo x: para baixo



Resumo dos principais parâmetros

- **eye** (x_e, y_e, z_e)
- **center** (x_c, y_c, z_c)
- **up** $(0, 1, 0)$
- **FOV** (ex.: 60°)
- **aspect ratio** (ex.: 16:9)
- z_{near}, z_{far}

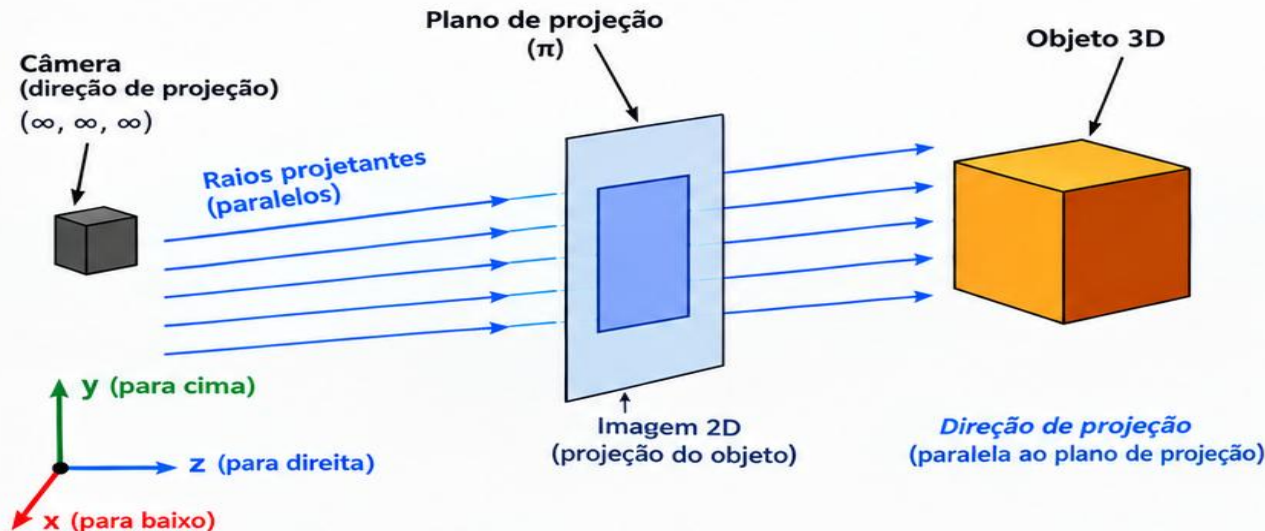
Características

- Linhas paralelas convergem em um ponto de fuga.
- Simula a visão humana.
- Produz sensação de profundidade.
- Mais realista que a projeção ortográfica.

A projeção perspectiva transforma coordenadas 3D em coordenadas 2D, considerando a posição da câmera, o campo de visão e os planos de recorte, produzindo a sensação de profundidade.

Sistema de Projeção Paralela (Ortográfica)

A projeção paralela (ortográfica) simula a visão de uma câmera infinita, onde os raios projetantes são paralelos entre si, mantendo as proporções dos objetos (sem ponto de fuga).



Parâmetros da Visualização

Plano de projeção (π)

Plano onde a imagem é formada.

(π : $x = 0$, por exemplo)

Posição da câmera (ou direção de projeção)

Define a direção dos raios projetantes.

(direção: $(0, 0, -1)$, por exemplo)

Ponto de referência (center / look-at)

Ponto para o qual a câmera está apontada.

(x_c, y_c, z_c)

Plano de corte (clipping)

Define a região visível do objeto.

(geralmente um volume de visualização ortogonal)

Plano de corte próximo e distante

Limitam a profundidade da cena.

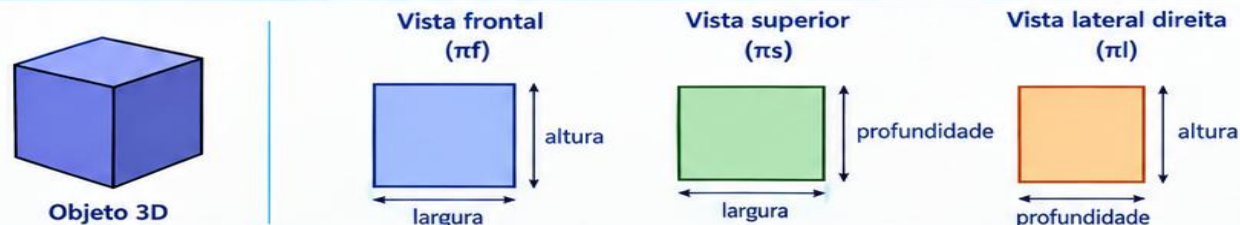
(z_{near}, z_{far})

Plano de corte de visualização (frustum ortográfico)

No caso ortográfico, é um paralelepípedo

(alinhado aos eixos).

Vistas ortográficas (exemplo: cubo)



Sistema de coordenadas (vista geral)

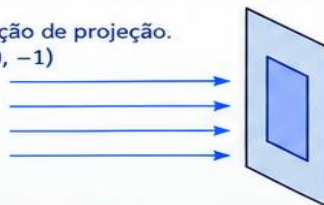


Eixos do sistema:

- y: para cima
- z: para direita
- x: para baixo

Raios projetantes (paralelos)

Direção de projeção.
($0, 0, -1$)



Resumo dos principais parâmetros

- π : plano de projeção (ex.: $x = 0$)
- (x_c, y_c, z_c) : ponto de referência
- $(0, 0, -1)$: direção de projeção
- z_{near}, z_{far} : planos de corte
- volume ortográfico: (l, r, b, t, n, f)

Características

- Linhas paralelas permanecem paralelas.
- Preserva as proporções do objeto.
- Não há ponto de fuga.
- Muito utilizada em desenhos técnicos e CAD.

Na projeção paralela (ortográfica), o objeto é representado por suas vistas em planos perpendiculares, com raios projetantes paralelos, mantendo as dimensões e proporções.

Perspectiva por semelhança de triângulos

- O plano de projeção fica a uma distância n da câmera
- O triângulo do ponto é semelhante ao do plano de projeção

Semelhança e fórmula

$$x_p / n = x_c / z_c$$

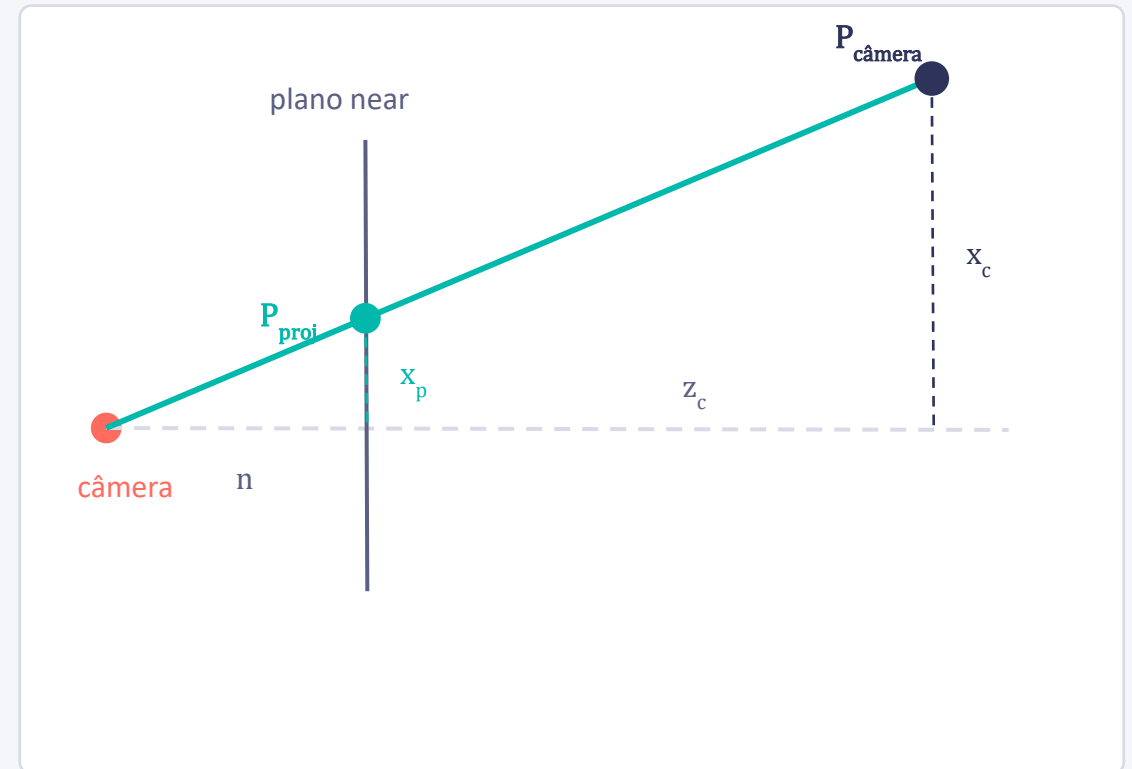
$$x_p = n \cdot x_c / z_c$$

$$y_p = n \cdot y_c / z_c$$

onde

n distância do plano near (projeção)

x_p, y_p ponto projetado no plano near



EXERCÍCIO

$$n = 1 \rightarrow x_p = 1 \cdot 4 / 8 = 0.5 \quad y_p = 1 \cdot 2 / 8 = 0.25$$

Por que a projeção precisa de w

- Dividir por z_c não é operação linear — nenhuma matriz sozinha faz isso
- A matriz então guarda z_c na coordenada w, e a divisão vem depois
- Esse passo separado é a divisão homogênea, feita pelo hardware
- É por isso que o recorte 3D acontece antes da divisão

1 — multiplicação matricial

$$P_{\text{clip}} = P_{\text{câmera}} \cdot P \quad \text{com } w' = z_c$$

2 — divisão homogênea

$$P_{\text{ndc}} = (x'/w', y'/w', z'/w')$$

w'

quarta coordenada, que recebe z_c

P_{clip}

ponto no espaço de recorte

A matriz de projeção P

Forma simbólica (vetor-linha)

$$P = \begin{bmatrix} n/\text{right} & 0 & 0 & 0 \\ 0 & n/\text{top} & 0 & 0 \\ 0 & 0 & (f+n)/(f-n) & 1 \\ 0 & 0 & -2fn/(f-n) & 0 \end{bmatrix}$$

onde

P	matriz de projeção perspectiva
n, f	planos near e far
right, top	meia-largura e meia-altura no near
o 1 na 3ª linha	copia z_c para w'

EXERCÍCIO

$$P = \begin{bmatrix} 0.75 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1.0202 & 1 \\ 0 & 0 & -2.0202 & 0 \end{bmatrix}$$

$n = 1, f = 100, \text{right} = 4/3, \text{top} = 1$

$$(4, 2, 8, 1) \cdot P = (3, 2, 6.14, 8)$$

$$/w' = 8 \rightarrow (0.375, 0.25, 0.77)$$

O volume visível da câmera

- Frustum: a pirâmide truncada que delimita o que a câmera enxerga
- Quatro parâmetros definem suas dimensões

Dimensões do plano near

$\text{aspect} = \text{largura} / \text{altura}$

$\text{top} = n \cdot \tan(\text{FOV}_y / 2)$

$\text{right} = \text{top} \cdot \text{aspect}$

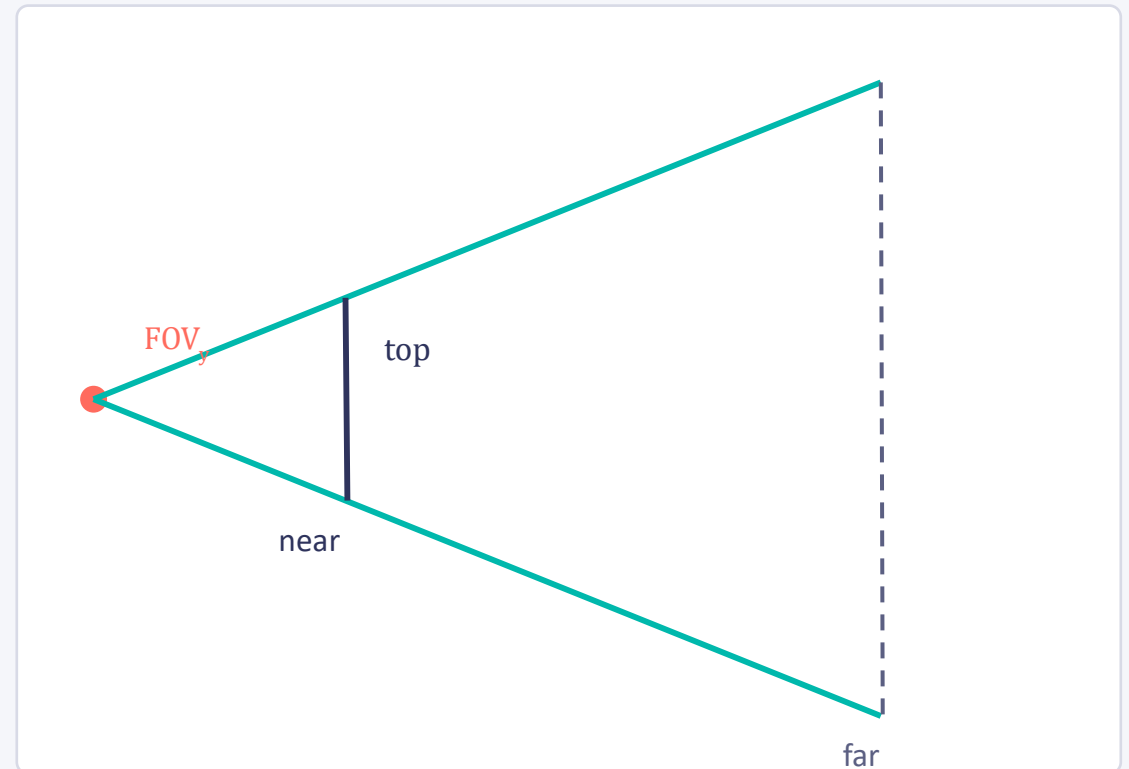
onde

FOV_y

abertura vertical da câmera

aspect

razão largura / altura da janela



EXERCÍCIO

$\text{aspect} = 800/600 = 4/3$ $\text{top} = 1 \cdot \tan(45^\circ) = 1$ $\text{right} = 4/3$

Cubo canônico NDC

Normalized Device Coordinates

- A matriz de projeção mapeia o frustum inteiro no cubo $[-1, 1]^3$
- Normalizar desacopla recorte e viewport da resolução da tela

Normalização

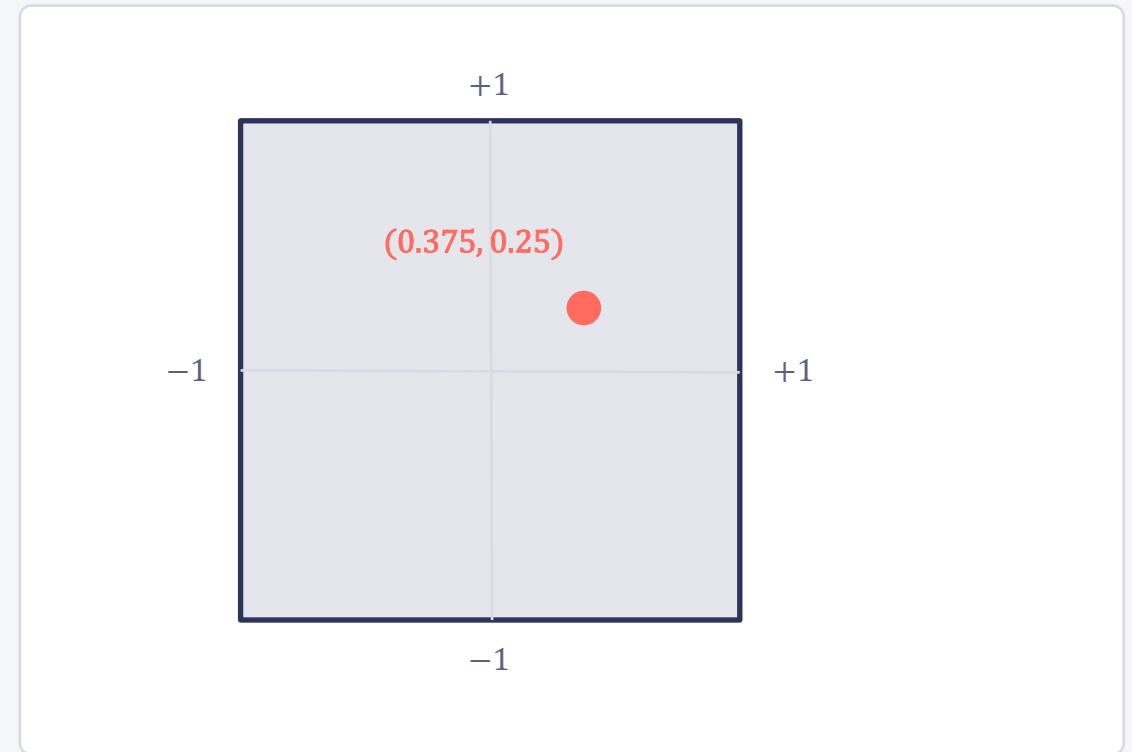
$$x_{\text{ndc}} = x_p / \text{right}$$

$$y_{\text{ndc}} = y_p / \text{top}$$

onde

x_{ndc} , y_{ndc}

coordenadas normalizadas em $[-1, 1]$



EXERCÍCIO

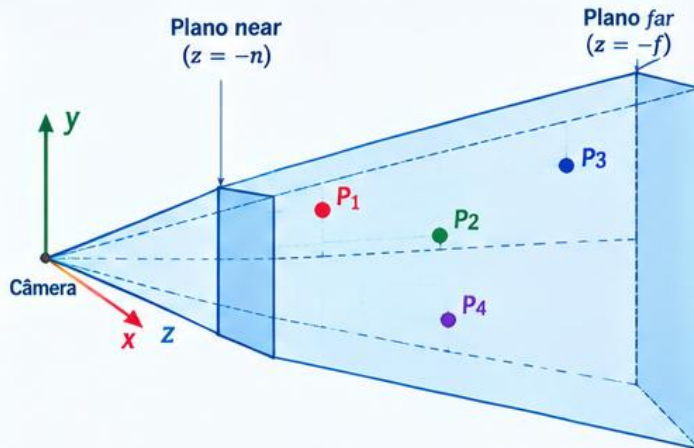
$$x_{\text{ndc}} = 0.5 / (4/3) = 0.375$$

$$y_{\text{ndc}} = 0.25 / 1 = 0.25$$

Mapeamento do Frustum para o Cubo Canônico (NDC)

Após a projeção, os pontos dentro do frustum de visualização são mapeados para o cubo canônico de coordenadas normalizadas (NDC), com valores no intervalo $[-1, 1]$ em x , y e z .

Frustum de visualização (espaço de câmera)



Atenção:
near e far são
negativos
nesta
imagem

Transformação de perspectiva → NDC

Para cada ponto (x, y, z) do espaço de câmera, o mapeamento para o cubo canônico é dado por:

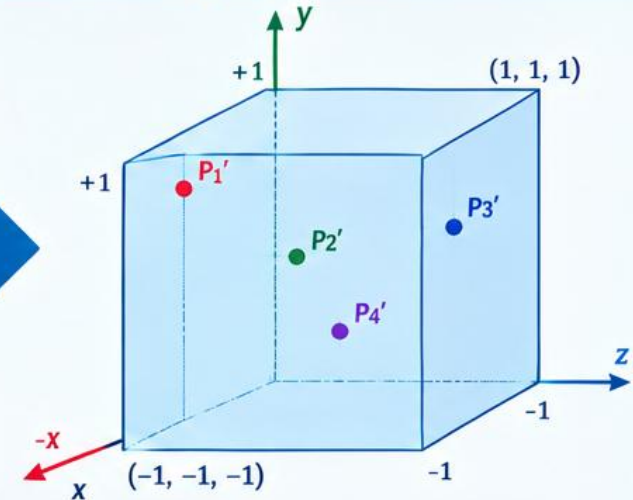
$$x_{ndc} = \frac{x}{-z} \cdot \frac{n}{r}$$

$$y_{ndc} = \frac{y}{-z} \cdot \frac{n}{t}$$

$$z_{ndc} = \frac{z + n}{f - n} \cdot 2 - 1$$

n = distância do plano *near* (positivo)
 f = distância do plano *far* (positivo)
 r = largura do plano *near* (metade da largura)
 t = altura do plano *near* (metade da altura)

Cubo Canônico (NDC)



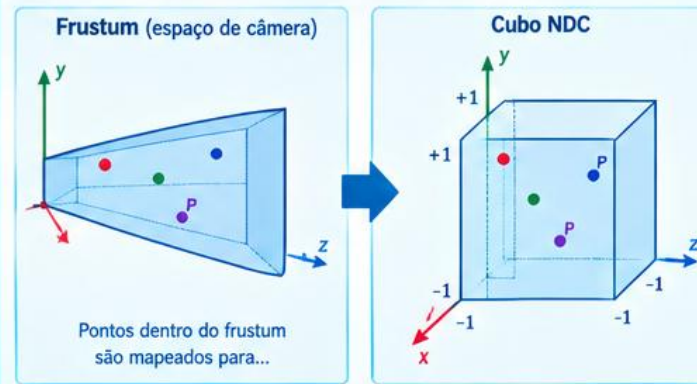
Exemplo de pontos no frustum e seu mapeamento

Ponto	Coordenadas no espaço de câmera (x, y, z)	Coordenadas em NDC ($x_{ndc}, y_{ndc}, z_{ndc}$)
● P_1	$(-0,5n, 0,5n, -n)$	$(-0,5, 0,5, -1)$
● P_2	$(0,0, 0,0, -(n+f)/2)$	$(0,0, 0,0, 0)$
● P_3	$(0,5n, 0,5n, -f)$	$(0,5, 0,5, 1)$
● P_4	$(-0,5n, -0,5n, -(n+f)/2)$	$(-0,5, -0,5, 0)$

Passo a passo (exemplo: P_1)

- Ponto no espaço de câmera:
 $P_1 = (-0,5n, 0,5n, -n)$
- Aplicar a projeção perspectiva (e depois a divisão por w):
$$x' = \frac{x}{-z} \cdot \frac{n}{r} \rightarrow x' = \frac{-0,5n}{n} \cdot \frac{n}{r} = -0,5r$$
$$y' = \frac{y}{-z} \cdot \frac{n}{t} \rightarrow y' = \frac{0,5n}{n} \cdot \frac{n}{t} = 0,5t$$
$$z' = \frac{z + n}{f - n} \cdot 2 - 1 \rightarrow z' = -1$$
- Resultado em NDC:
 $P_1' = (-0,5, 0,5, -1)$

Resultado visual



Resumo

- O frustum é mapeado para o cubo canônico NDC com coordenadas no intervalo $[-1, 1]$.
- O mapeamento usa as dimensões do plano *near* (n, r, t) e dos planos *near* e *far* (n, f).
- Após o mapeamento, os pontos estão prontos para a etapa de viewport (mapeamento para a janela de tela).

NDC: $x, y, z \in [-1, 1]$
(cubo canônico)

03

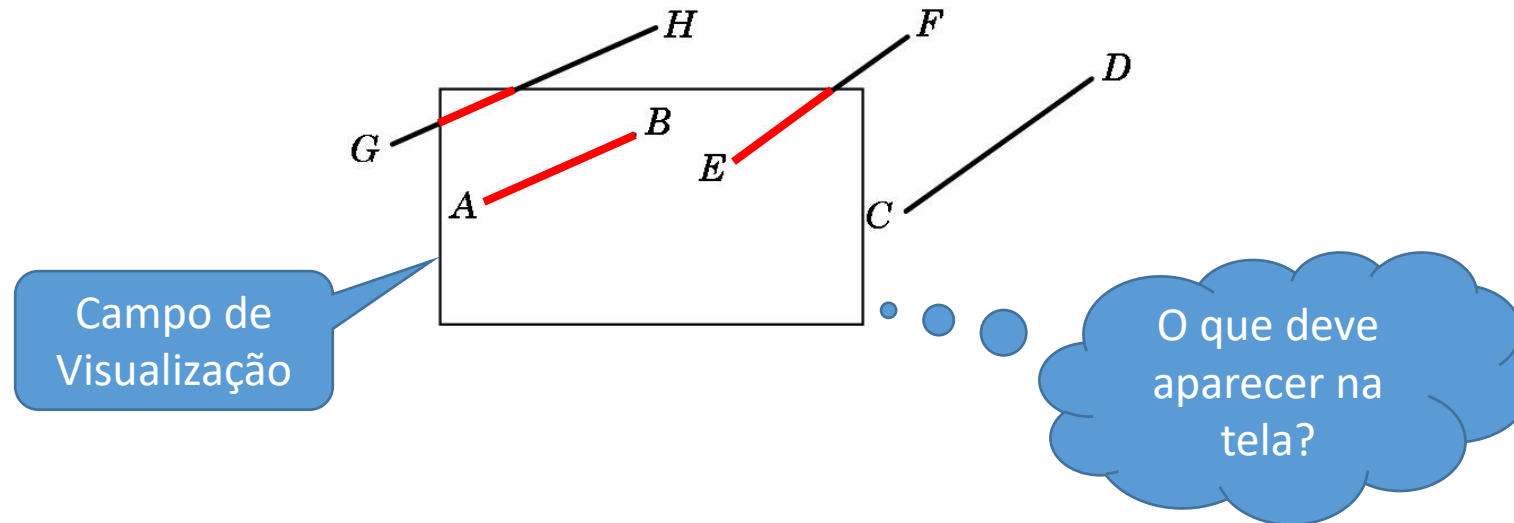
PARTE 3 DE 4

Recorte (Clipping)

Descartando geometria invisível antes de gastar processamento

Algoritmo de Recorte (Clipping)

Um **algoritmo de *clipping*** é um método utilizado em **computação gráfica** para determinar qual parte de uma linha está dentro ou fora de uma **janela de visualização retangular**.

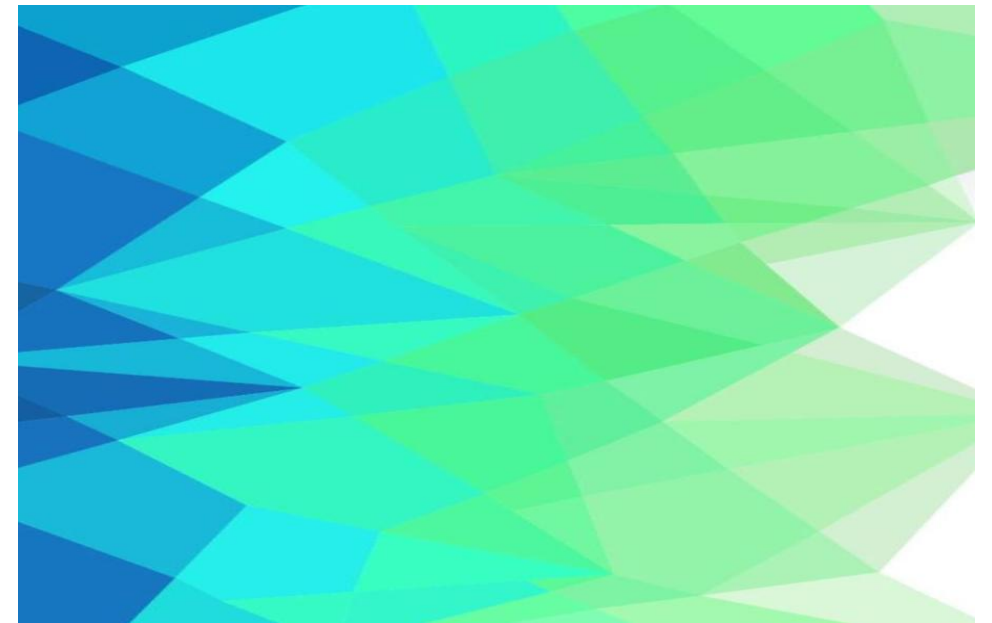


Clipping

Objetivo

Dado um **segmento de linha** e uma **janela retangular de recorte**, o algoritmo determina:

- Se a linha está **totalmente dentro**, a linha é aceita;
- Se a linha está **totalmente fora**, a linha é rejeitada;
- Senão, calcula-se os pontos de interseção com as bordas da janela para *recortar* a linha;
- Se **cruza** as bordas da janela na parte de dentro da janela, calcula-se o ponto da borda.

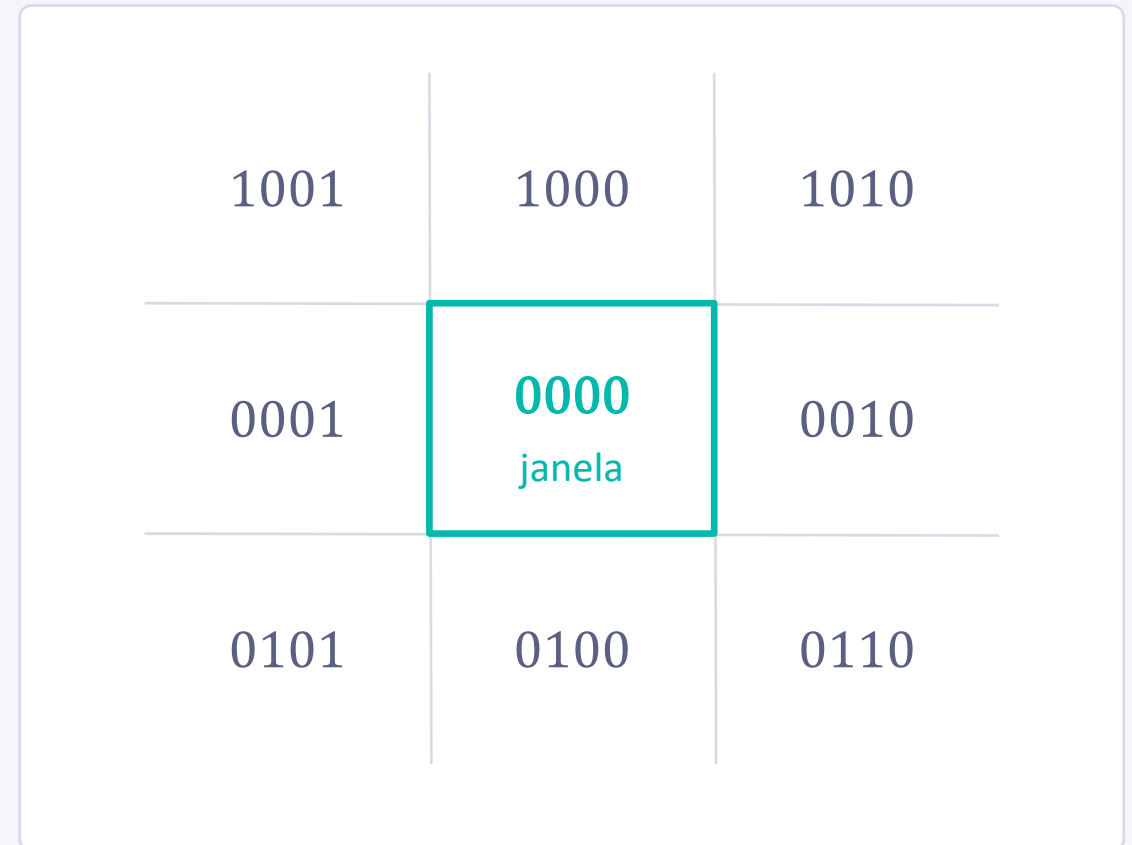


Cohen-Sutherland: códigos de região

- O plano é dividido em 9 regiões ao redor da janela de recorte
- Cada ponto recebe um código de 4 bits, um por pergunta

Atribuição dos bits

bit 0 (0001) : $x < x_{\min}$ esquerda
bit 1 (0010) : $x > x_{\max}$ direita
bit 2 (0100) : $y < y_{\min}$ abaixo
bit 3 (1000) : $y > y_{\max}$ acima



Aceitação e rejeição trivial

$\text{cód}(A) \text{ OR } \text{cód}(B) = 0$

Os dois pontos estão dentro $\rightarrow$ aceita o segmento inteiro

$\text{cód}(A) \text{ AND } \text{cód}(B) \neq 0$

Os dois estão fora do mesmo lado $\rightarrow$ rejeita tudo

caso contrário

Calcula a interseção com a aresta e repete o teste

EXERCÍCIO

$0.375 \in [-1,1] \rightarrow \text{bits } x = 00$ $0.25 \in [-1,1] \rightarrow \text{bits } y = 00$
código = 0000 $\rightarrow$ aceitação trivial: o ponto é visível

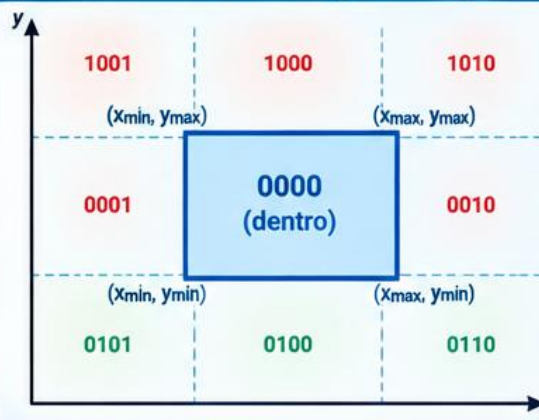
Algoritmo Cohen-Sutherland – Recorte de Linhas

O algoritmo Cohen-Sutherland é um método eficiente para recortar segmentos de reta a um retângulo de visualização (janela de recorte), utilizando códigos de região e testes lógicos, evitando cálculos de interseção desnecessários.

Conceito e ideia principal

- Cada ponto é associado a um código de região (4 bits) que indica sua posição em relação à janela de recorte.
- Se o código lógico (AND) dos códigos das extremidades for 0, o segmento pode estar dentro da janela (ou cruza a janela).
- Se o código lógico (AND) for diferente de 0, o segmento está totalmente fora da janela.
- Caso contrário, calcula-se o ponto de interseção com a borda e repete-se o processo.

Janela de recorte e códigos de região



Códigos de recorte (4 bits)	
Bit	Significado
b_3 (1000)	Acima (Top)
b_2 (0100)	Abaixo (Bottom)
b_1 (0010)	À direita (Right)
b_0 (0001)	À esquerda (Left)

Exemplo: ponto (x, y)
Código = $b_3 b_2 b_1 b_0$

Teste lógico e decisão

1. Rejeição (fora da janela)

Se $(c_1 \text{ AND } c_2) \neq 0$
→ segmento totalmente fora (rejeitar).

2. Aceitação (dentro da janela)

Se $(c_1 \text{ OR } c_2) = 0$
→ segmento totalmente dentro (aceitar).

3. Caso geral (recorte)

Se as condições acima não forem verdadeiras
→ calcular o ponto de interseção com a borda e atualizar o ponto externo.

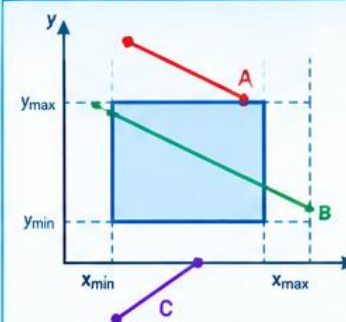
Passo a passo do algoritmo

- Calcule o código de região para cada extremidade do segmento (p_1 e p_2).
- Faça o teste lógico (AND e OR).
- Se $(c_1 \text{ AND } c_2) \neq 0$ → rejeite.
- Se $(c_1 \text{ OR } c_2) = 0$ → aceite (segmento dentro).
- Caso contrário, determine qual borda é intersectada (com base no código da extremidade externa).
- Calcule o ponto de interseção, substitua o ponto externo e volte ao passo 2.

Determinação da borda de interseção

Código da extremidade externa (exemplo)	Borda a ser testada	Cálculo do ponto de interseção
1000 (top)	$y = y_{\max}$	$x = x_1 + (x_2 - x_1) \frac{(y_{\max} - y_1)}{(y_2 - y_1)}$
0100 (bottom)	$y = y_{\min}$	$x = x_1 + (x_2 - x_1) \frac{(y_{\min} - y_1)}{(y_2 - y_1)}$
0010 (right)	$x = x_{\max}$	$y = y_1 + (y_2 - y_1) \frac{(x_{\max} - x_1)}{(x_2 - x_1)}$
0001 (left)	$x = x_{\min}$	$y = y_1 + (y_2 - y_1) \frac{(x_{\min} - x_1)}{(x_2 - x_1)}$

Exemplo de funcionamento



Códigos dos pontos

A: (10, 90) → 1000 (top)
(80, 20) (inside)
B: (20, 50) → 0000 (inside)
(120, 80) → 0010 (right)
C: (10, 10) → 0100 (bottom)
(60, 30) → 0000 (inside)

Resultado

- A: rejeitado (1000 AND 0000 = 0000? → não, mas 1000 AND 0000 = caso geral e é recortado até a borda superior, então não até a borda superior).
- B: recortado (cruza a janela).
- C: recortado (cruza a janela).

Pseudocódigo (simplificado)

```
function clipLine(x1, y1, x2, y2):
    c1 = computeCode(x1, y1)
    c2 = computeCode(x2, y2)
    while True:
        if (c1 AND c2) != 0:
            return REJECT
        if (c1 OR c2) == 0:
            return ACCEPT
        if c1 != 0:
            (x, y) = intersectWithBoundary(x1, y1, x2, y2)
            c1 = computeCode(x, y)
            x1, y1 = x, y
        else:
            (x, y) = intersectWithBoundary(x1, y1, x2, y2)
            c2 = computeCode(x, y)
            x2, y2 = x, y
```

✓ Vantagens

- Evita cálculos de interseção desnecessários.
- Simples e eficiente.
- Funciona bem para grandes quantidades de segmentos.

! Limitações

- Menos eficiente para recortes complexos (ex.: polígonos).
- Não lida diretamente com recortes de curvas.

Polígonos e recorte no espaço homogêneo

Sutherland-Hodgman

Recorta o polígono contra uma aresta por vez. Quatro casos por segmento:

dentro → dentro mantém o vértice final

dentro → fora gera a interseção

fora → dentro interseção + vértice final

fora → fora descarta

Recorte no espaço homogêneo

- Em 3D o recorte ocorre antes da divisão por w
- A visibilidade vira seis desigualdades simétricas
- Recortar antes evita erros com pontos atrás da câmera

Os seis planos do frustum

$$-w \leq x \leq w$$

$$-w \leq y \leq w$$

$$-w \leq z \leq w$$

Clipping Sutherland- Hodgeman



Entrada e Saída:

Entrada: lista de vértices de polígonos em ordem

Saída: lista de vértices de polígonos recortados consistindo de vértices velhos (talvez) e novos (talvez)



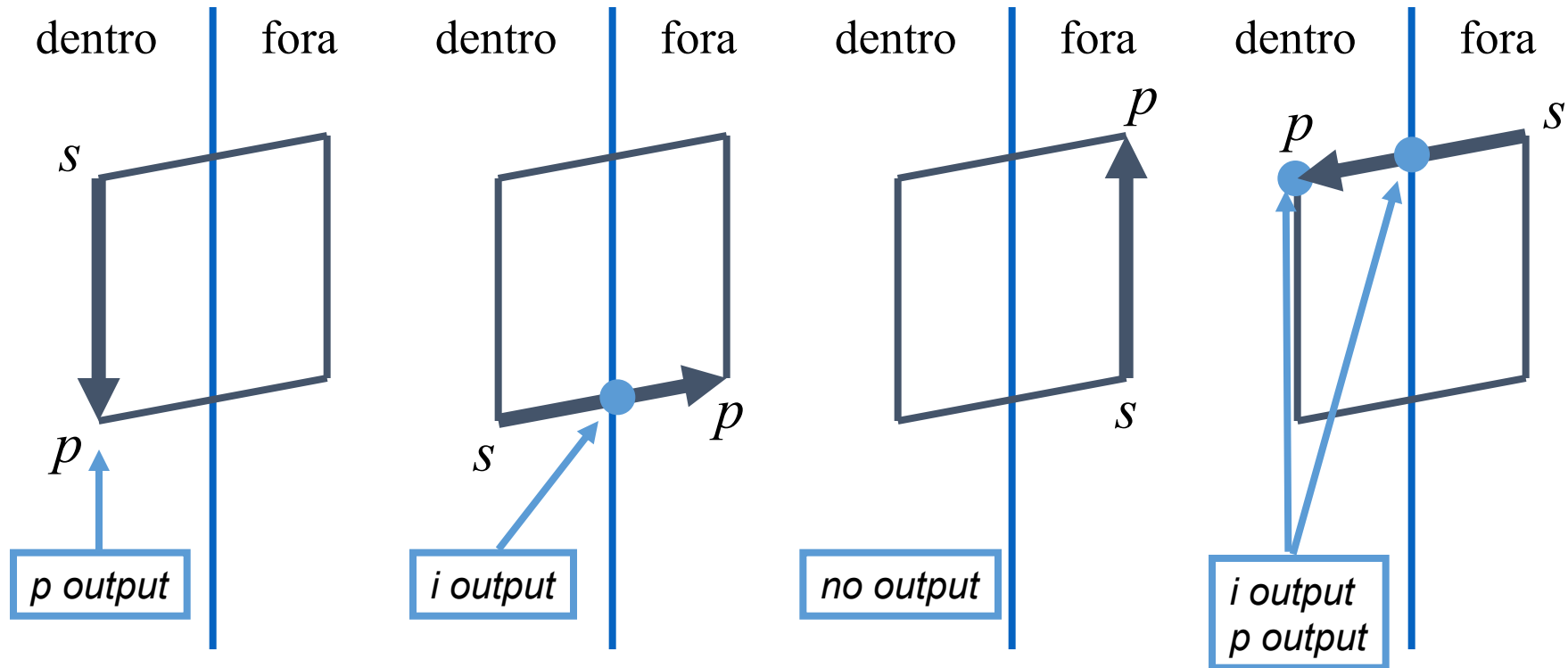
Rotina básica:

Caminha em volta do polígono um vértice por vez

Vértice corrente tem posição p

Vértice anterior tinha posição s , e ele é adicionado para a saída se apropriado

Clipping Sutherland-Hodgeman



04

PARTE 4 DE 4

Transformação de Viewport

O último passo: do cubo canônico aos pixels da janela

De coordenadas normalizadas a pixels

- NDC vive em $[-1,1]$; a janela tem largura e altura em pixels
- O eixo Y é invertido: NDC cresce para cima, a tela para baixo

Forma direta

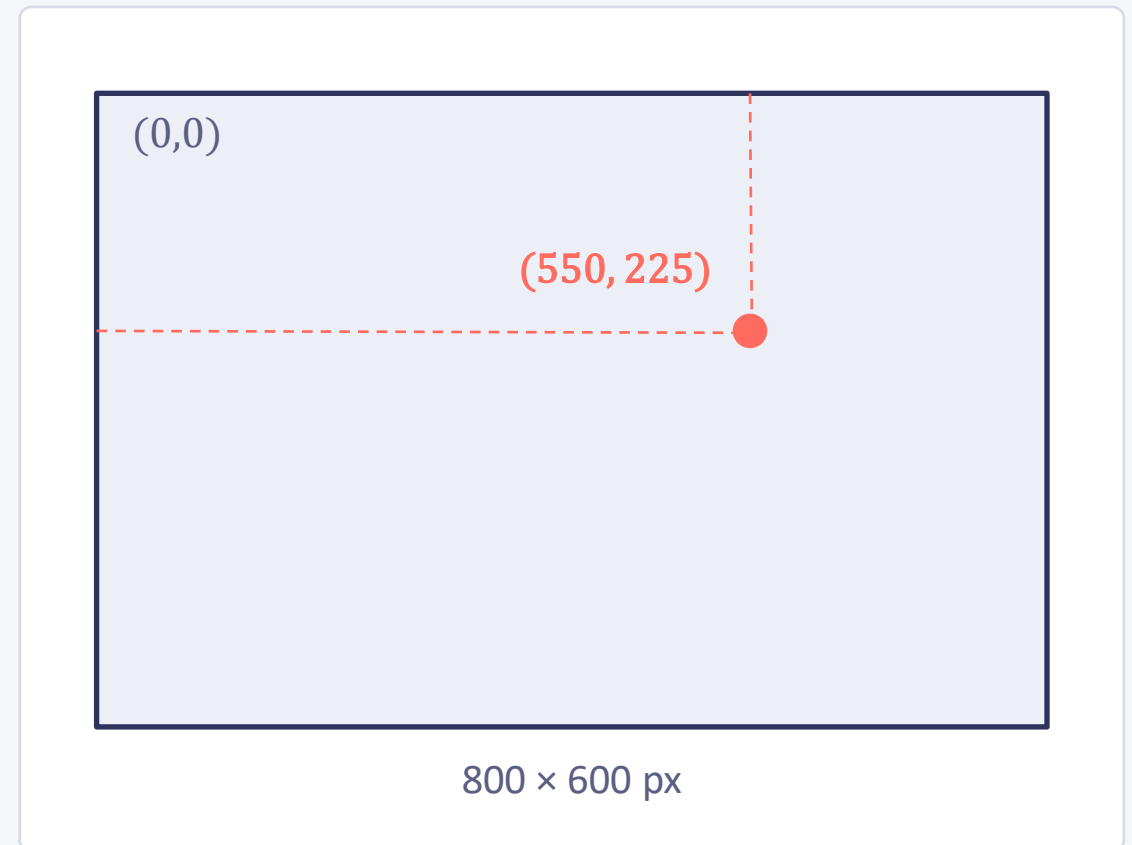
$$x_{\text{tela}} = (x_{\text{ndc}} + 1) / 2 \cdot \text{largura}$$

$$y_{\text{tela}} = (1 - y_{\text{ndc}}) / 2 \cdot \text{altura}$$

onde

largura, altura dimensões da janela em pixels

$x_{\text{tela}}, y_{\text{tela}}$ posição final do pixel



EXERCÍCIO

$$x_{\text{tela}} = (0.375 + 1) / 2 \cdot 800 = 550 \quad y_{\text{tela}} = (1 - 0.25) / 2 \cdot 600 = 225$$

A matriz de viewport W

Forma simbólica (vetor-linha)

$$W = \begin{bmatrix} \text{largura}/2 & 0 & 0 & 0 \\ 0 & -\text{altura}/2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ \text{largura}/2 & \text{altura}/2 & 0 & 1 \end{bmatrix}$$

onde

W	matriz de viewport 4 × 4
diagonal	faz a escala de NDC para pixels
última linha	desloca a origem para o canto
−altura/2	é o que inverte o eixo Y

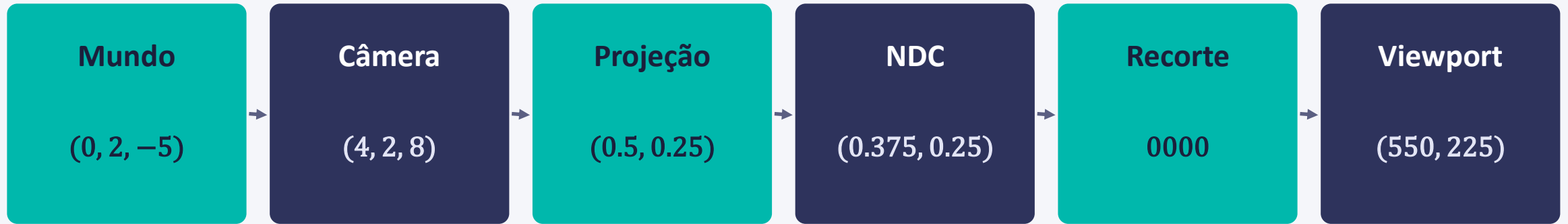
EXERCÍCIO

$$W = \begin{bmatrix} 400 & 0 & 0 & 0 \\ 0 & -300 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 400 & 300 & 0 & 1 \end{bmatrix}$$

largura = 800, altura = 600

$$\begin{aligned} & (0.375, 0.25, z, 1) \cdot W \\ & = (550, 225, z, 1) \end{aligned}$$

A jornada completa, do mundo ao pixel



- É sempre o mesmo ponto — muda só o sistema de coordenadas em que ele é expresso
- A câmera fora dos eixos exige construir a base $(\mathbf{f}, \mathbf{r}, \mathbf{u})$; alinhada, bastaria transladar
- A divisão por z_c é a única etapa não linear de todo o caminho

Compondo as matrizes: passo 1 — $V \cdot P$

Na convenção vetor-linha as matrizes se multiplicam na ordem em que são aplicadas: primeiro V, depois P, depois W.

$$\begin{array}{c} V \\ \begin{bmatrix} 0.6 & 0 & -0.8 & 0 \\ 0 & 1 & 0 & 0 \\ -0.8 & 0 & -0.6 & 0 \\ 0 & 0 & 5 & 1 \end{bmatrix} \end{array} \cdot \begin{array}{c} P \\ \begin{bmatrix} 0.75 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1.0202 & 1 \\ 0 & 0 & -2.0202 & 0 \end{bmatrix} \end{array} = \begin{array}{c} V \cdot P \\ \begin{bmatrix} 0.45 & 0 & -0.8162 & -0.8 \\ 0 & 1 & 0 & 0 \\ -0.6 & 0 & -0.6121 & -0.6 \\ 0 & 0 & 3.0808 & 5 \end{bmatrix} \end{array}$$

Cada elemento é o produto escalar de uma linha de V por uma coluna de P.

Exemplo: $(0.6)(0.75) + (0)(0) + (-0.8)(0) + (0)(0) = 0.45$

Passo 2 — $M = (V \cdot P) \cdot W$

$$\begin{array}{c} V \cdot P \\ \left[\begin{array}{cccc} 0.45 & 0 & -0.8162 & -0.8 \\ 0 & 1 & 0 & 0 \\ -0.6 & 0 & -0.6121 & -0.6 \\ 0 & 0 & 3.0808 & 5 \end{array} \right] \end{array} \cdot \begin{array}{c} W \\ \left[\begin{array}{cccc} 400 & 0 & 0 & 0 \\ 0 & -300 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 400 & 300 & 0 & 1 \end{array} \right] \end{array} = \begin{array}{c} M \\ \left[\begin{array}{cccc} -140 & -240 & -0.8162 & -0.8 \\ 0 & -300 & 0 & 0 \\ -480 & -180 & -0.6121 & -0.6 \\ 2000 & 1500 & 3.0808 & 5 \end{array} \right] \end{array}$$

EXERCÍCIO

$$P_{\text{tela}} = P_{\text{mundo}} \cdot M = (0, 2, -5, 1) \cdot M = (4400, 1800, 6.14, 8)$$

divisão homogênea: $/ w' = 8 \rightarrow (550, 225, 0.77)$

Uma única matriz M leva o ponto do mundo direto ao pixel — pré-multiplicada uma vez por quadro.

Pipeline Gráfica – Cálculo da Matriz $M = V \cdot P \cdot W$ (Vetor Linha)

No modelo com **vetor linha**, a transformação é feita da esquerda para a direita.

$$p' = p \cdot M$$

$$M = V \cdot P \cdot W$$

Coordenadas do objeto

$(x, y, z, 1)$

Matriz de visualização

V
(câmera)

Matriz de projeção

P
(clip)

Matriz de viewport

W
(tela)

Coordenadas na tela

$(x, y, z, 1)$
(em pixels)

1. Matriz de Visualização (V)

Transforma o sistema de coordenadas do mundo para o sistema da câmera.

Para vetor linha, a matriz é a transposta da forma usual (vetor coluna).

$$V = \begin{bmatrix} r_x & r_y & r_z & 0 \\ u_x & u_y & u_z & 0 \\ -f_x & -f_y & -f_z & 0 \\ -r \cdot e & -u \cdot e & f \cdot e & 1 \end{bmatrix}$$

r = vetor da direita (right)

u = vetor para cima (up)

f = vetor para frente (forward)

e = posição da câmera (ex.: e_x, e_y, e_z)

2. Matriz de Projeção (P)

Projeta o espaço da câmera no volume de recorte (clip).

Para vetor linha, a matriz é a transposta da forma usual.

$$P = \begin{bmatrix} q/a & 0 & 0 & 0 \\ 0 & q & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & -1 \\ 0 & 0 & \frac{2fn}{n-f} & 0 \end{bmatrix}$$

$$q = \frac{1}{\tan(fov/2)} \quad (\text{fov} = \text{campo de visão vertical})$$

a = razão de aspecto (largura/altura)

n = plano próximo (near)

f = plano distante (far)

3. Matriz de Viewport (W)

Transforma o espaço de clip em coordenadas de tela.

Para vetor linha, a matriz é a transposta da forma usual.

$$W = \begin{bmatrix} w/2 & 0 & 0 & x_0 + w/2 \\ 0 & h/2 & 0 & y_0 + h/2 \\ 0 & 0 & 1/2 & z_0 + 1/2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

w = largura da janela de visualização (em pixels)

h = altura da janela de visualização (em pixels)

x_0, y_0 = posição da janela (origem)

z_0 = valor de profundidade (geralmente 0)

Matriz Combinada

A matriz final é o produto das três matrizes na ordem:

$$M = V \cdot P \cdot W$$

e a transformação dos pontos é dada por:

$$p' = p \cdot M$$

Dimensões e ordem das matrizes

- V : 4×4 (visualização)
- P : 4×4 (projeção)
- W : 4×4 (viewport)
- M : 4×4 (matriz combinada)

Exemplo numérico simplificado

(valores ilustrativos)

Suponha: $q = 1, a = 1, n = 1, f = 10$,

janela de 800×600 pixels, origem $(0, 0)$, $z_0 = 0$.

As matrizes ficam:

$$M = V \cdot P \cdot W = \begin{bmatrix} 400 & 0 & 0 & 400 \\ 0 & 300 & 0 & 300 \\ 0 & 0 & 0.5 & 0 \\ 0 & 0 & -0.5 & 1 \end{bmatrix}$$

Observações

- A matriz de viewport (W) depende do tamanho da janela e da posição na tela.
- Em muitos sistemas (ex.: OpenGL), a matriz de projeção e de visualização seguem a convenção de vetor coluna. Aqui foram apresentadas na forma de vetor linha (transpostas).
- Após a transformação, é necessário realizar a divisão por w (perspectiva):
 $(x, y, z) = (x'/w, y'/w, z'/w)$.

Convenções utilizadas que explicam as diferenças entre as matrizes W apresentadas

Diferença	Nossa versão (da aula)	Versão que você achou
Y	invertida (raster, y para baixo)	não invertida (Cartesiana, y para cima)
Viewport	sempre começa em (0,0)	pode começar em qualquer (x_min, y_min)
Z	passa direto (não usamos ainda)	mapeado para uma faixa configurável de profundidade

CONCLUSÕES

Do mundo 3D ao pixel na tela — uma cadeia de mudanças de referencial, resolvida por três matrizes e uma divisão.

O pipeline, em uma frase

- Mundo → Câmera → Projeção → NDC → Recorte → Viewport: sempre a mesma cadeia de espaços
- V, P e W descrevem a câmera e a tela — e se compõem numa única M por quadro
- Qualquer vértice vira pixel com uma multiplicação: $P_{\text{mundo}} \cdot M$

Os pontos que mais importam

- A divisão por w cria a perspectiva — por isso o recorte acontece antes dela
- NDC desacopla recorte e viewport da resolução da tela
- Clipping e rasterização decidem o que é visível e como cada pixel é pintado

Obrigado!

Dúvidas?

